

Artificial Intelligence

For HEDSPI Project

Lecturer 3 - Search

Lecturers :

Dr. Le Thanh Huong

Dr. Tran Duc Khanh

Dr. Hai V. Pham

Faculty of Information Technology - HUST

1

Outline

- Problem-solving agents
- Problem types
- Problem formulation
- Example problems
- Basic search algorithms
 - breadth-first search
 - depth-first search
 - depth-limited search
 - iterative deepening depth-first search

2
2

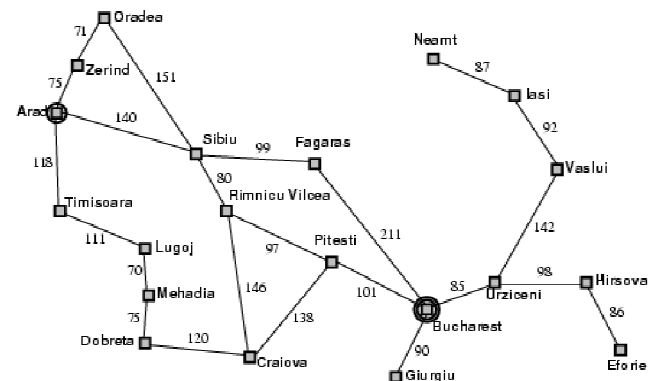
Problem-solving agents

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  static: seq, an action sequence, initially empty
           state, some description of the current world state
           goal, a goal, initially null
           problem, a problem formulation

  state ← UPDATE-STATE(state, percept)
  if seq is empty then do
    goal ← FORMULATE-GOAL(state)
    problem ← FORMULATE-PROBLEM(state, goal)
    seq ← SEARCH(problem)
  action ← FIRST(seq)
  seq ← REST(seq)
  return action
```

3
3

Example 1: Route Planning



- Performance: Get from Arad to Bucharest as quickly as possible
- Environment: The map, with cities, roads, and guaranteed travel times
- Actions: Travel a road between adjacent cities

4

Example 2: Finding letters

- Replace letters by numbers from 0 to 9 such as no different letter is replaced by the same number and satisfying the following constraint:

$$\begin{array}{r} \text{SEND} \\ + \text{MORE} \\ \hline \text{MONEY} \end{array} \quad \begin{array}{r} \text{CROSS} \\ + \text{ROADS} \\ \hline \text{DANGER} \end{array}$$

5

Example 3: Pouring water

- Given 2 containers A(m litres), B(n litres). Finding a method to measure k litres ($k \leq \max(m,n)$) by 2 containers A, B and a container C
- Actions (how):
 $C \rightarrow A; C \rightarrow B; A \rightarrow B; A \rightarrow C; B \rightarrow A; B \rightarrow C$
- Conditions: no overflow, pouring all water
- Eg: m = 5, n = 6, k = 2 (what)
- Mathematical model:
 $(x, y) \rightarrow (x', y')$
A B A B

6

Example 4: The 8-puzzle

- Trong bảng ô vuông n hàng, n cột, mỗi ô chứa 1 số nằm trong phạm vi từ $1 \rightarrow n^2 - 1$ sao cho không có 2 ô có cùng giá trị. Còn đúng 1 ô bị trống. Xuất phát từ 1 cách sắp xếp nào đó của các đồ của các số trong bảng, hãy dịch chuyển các ô trống sang phải, sang trái, lên trên, xuống dưới để đưa về bảng:

7	2	4
5		6
8	3	1

Start State

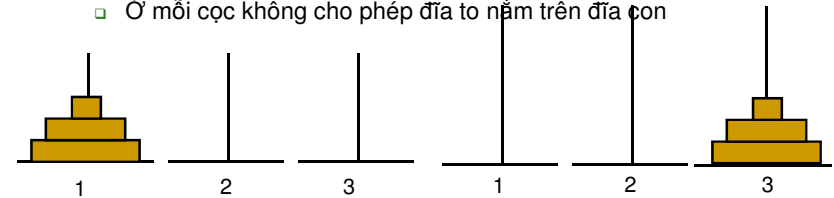
	1	2
3	4	5
6	7	8

Goal State

7

Example 5: Hà Nội tower

- Cho 3 cọc 1,2,3. Ở cọc 1 ban đầu có n đĩa, sắp theo thứ tự to dần từ trên xuống dưới. Hãy tìm cách chuyển n đĩa đó sang cọc 3 sao cho:
 - Mỗi lần chỉ chuyển 1 đĩa
 - Ở mỗi cọc không cho phép đĩa to nằm trên đĩa con



Bài toán tháp Hà Nội với $n = 3$

8

Problem types

- Deterministic, fully observable → single-state problem
 - Agent knows exactly which state it will be in; solution is a sequence
- Non-observable → sensorless problem (conformant problem)
 - Agent may have no idea where it is; solution is a sequence
- Nondeterministic and/or partially observable → contingency problem
 - percepts provide new information about current state
 - often interleave → search, execution
- Unknown state space → exploration problem

9

Search Problem Definition

A **problem** is defined by four items:

1. **initial state**: e.g., Arad
 2. **actions** or **successor function** $S(x)$ = set of action-state pairs
 - e.g., $S(\text{Arad}) = \{ \langle \text{Arad} \rightarrow \text{Zerind}, \text{Zerind} \rangle, \dots \}$
 3. **goal test**, can be
 - **explicit**, e.g., $x = \text{Bucharest}$
 - **implicit**, e.g., $\text{Checkmate}(x)$
 4. **path cost** (additive)
 - e.g., sum of distances, number of actions executed, etc.
 - $c(x, a, y)$ is the **step cost**, assumed to be ≥ 0
- A **solution** is a sequence of actions leading from the initial state to a goal state

10

Example: The 8-puzzle

7	2	4
5		6
8	3	1

Start State

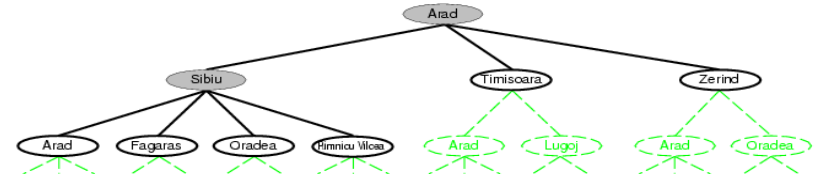
	1	2
3	4	5
6	7	8

Goal State

- states? locations of tiles
- actions? move blank left, right, up, down
- goal test? = goal state (given)
- path cost? 1 per move

11

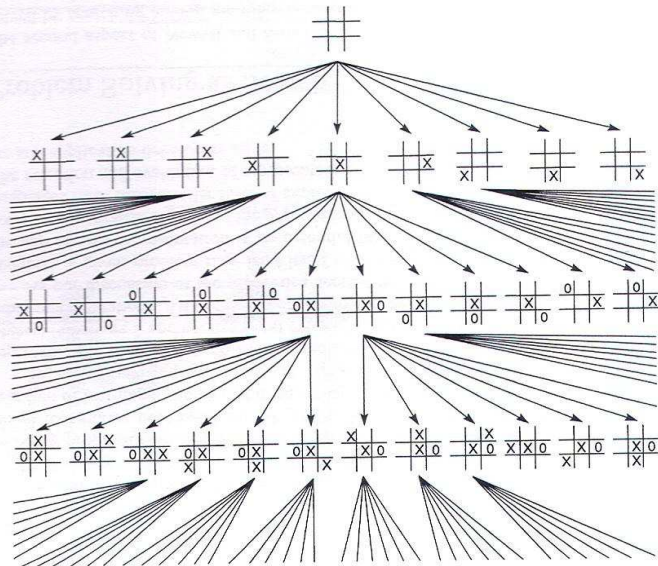
Search tree



- Search trees:
 - Represent the branching paths through a state graph.
 - Usually **much** larger than the state graph.
 - Can a finite state graph give an infinite search tree?

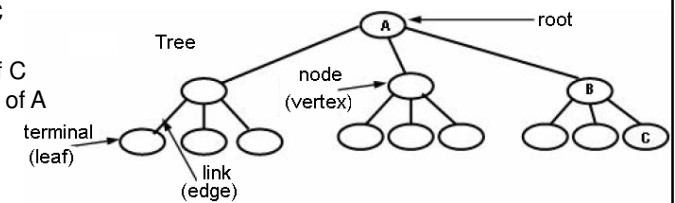
12

Search space of the game Tic-Tac-Toe

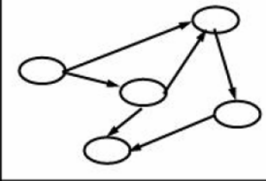


Tree and graph

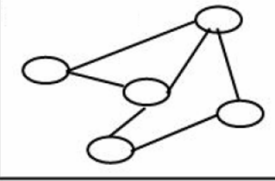
B is parent of C
C is child of B
A is ancestor of C
C is descendant of A



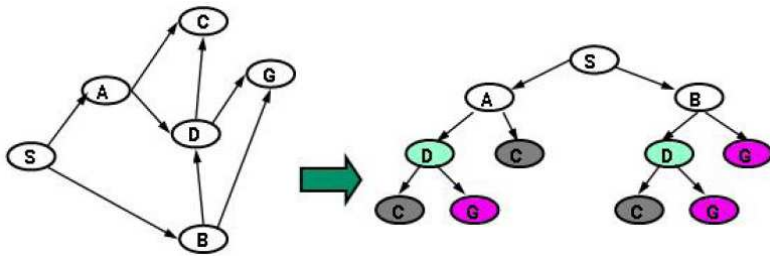
Directed graph
(one-way street)



Undirected graph
(two-way streets)



Convert from search graph to search tree



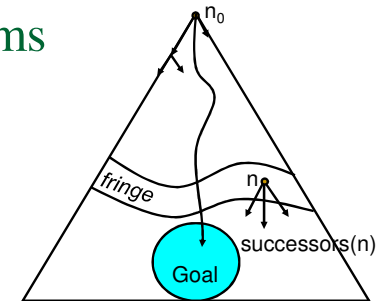
- We can turn graph search problems into tree search problems by:
 - replacing undirected links by 2 directed links
 - avoiding loops in path (or keeping track of visited nodes globally)

15

Tree search algorithms

■ Basic idea:

- offline, simulated exploration of state space by generating successors of already-explored states



```

function TREE-SEARCH(problem, strategy) returns a solution, or failure
    initialize the search tree using the initial state of problem
    loop do
        if there are no candidates for expansion then return failure
        choose a leaf node for expansion according to strategy
        if the node contains a goal state then return the corresponding solution
        else expand the node and add the resulting nodes to the search tree
    
```

16

Implementation: general tree search

```

function TREE-SEARCH( problem, fringe) returns a solution, or failure
  fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node ← REMOVE-FRONT(fringe)
    if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)
    fringe ← INSERT-ALL(EXPAND(node, problem), fringe)

```

```

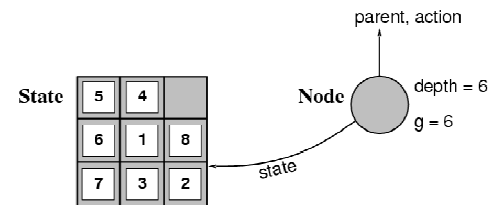
function EXPAND( node, problem) returns a set of nodes
  successors ← the empty set
  for each action, result in SUCCESSOR-FN[problem](STATE[node]) do
    s ← a new NODE
    PARENT-NODE[s] ← node; ACTION[s] ← action; STATE[s] ← result
    PATH-COST[s] ← PATH-COST[node] + STEP-COST(node, action, s)
    DEPTH[s] ← DEPTH[node] + 1
    add s to successors
  return successors

```

17

Implementation: states vs. nodes

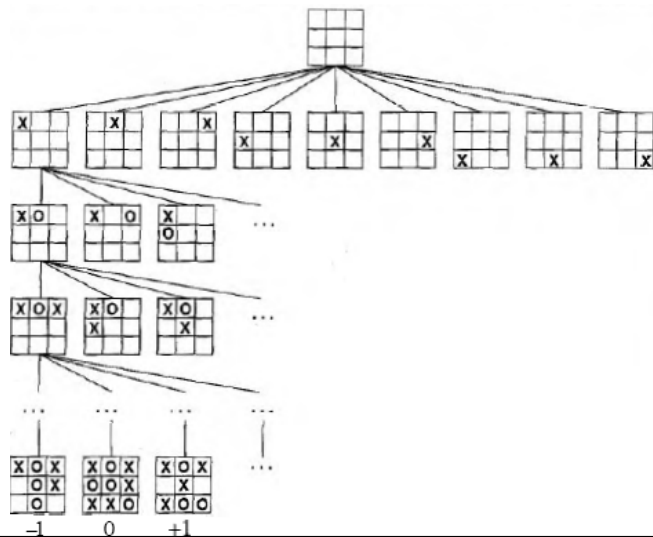
- A **state** is a (representation of) a physical configuration
- A **node** is a data structure constituting part of a search tree includes **state**, **parent node**, **action**, **path cost** $g(x)$, **depth**



- The **Expand** function creates new nodes, filling in the various fields and using the **SuccessorFn** of the problem to create the corresponding states.

18

Implementation: states vs. nodes



19

Search strategies

- A search strategy is defined by picking the **order of node expansion**
- Strategies are evaluated along the following dimensions:
 - **completeness**: does it always find a solution if one exists?
 - **time complexity**: number of nodes generated
 - **space complexity**: maximum number of nodes in memory
 - **optimality**: does it always find a least-cost solution?
- Time and space complexity are measured in terms of
 - b : maximum branching factor of the search tree
 - d : depth of the least-cost solution
 - m : maximum depth of the state space (may be ∞)

20

Uninformed search strategies

- **Uninformed** search strategies use only the information available in the problem definition
- Breadth-first search
 - Expand shallowest unexpanded node
 - *fringe* = queue (FIFO)

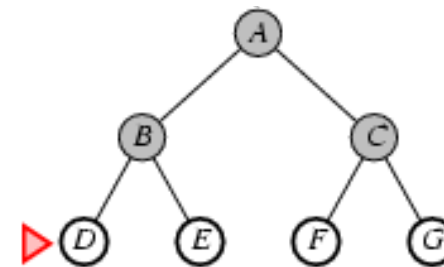
$out \leftarrow \frac{n \text{ fringe} \text{ successors}(n)}{in}$
- Depth-first search
 - Expand deepest unexpanded node
 - *fringe* = stack (LIFO)

$\frac{successors(n)}{in} \rightarrow out \leftarrow n \text{ fringe}$
- Depth-limited search: depth-first search with depth limit
- Iterative deepening search

21

Breadth-first search

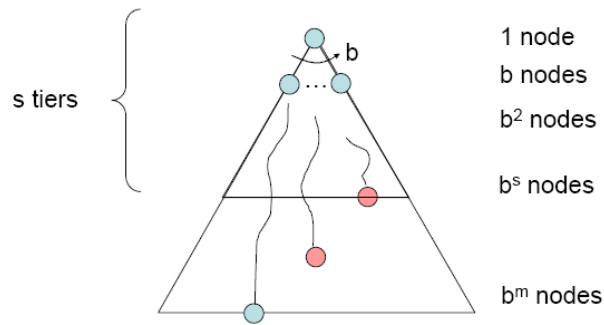
- Expand shallowest unexpanded node



22

Breadth-first search (con't)

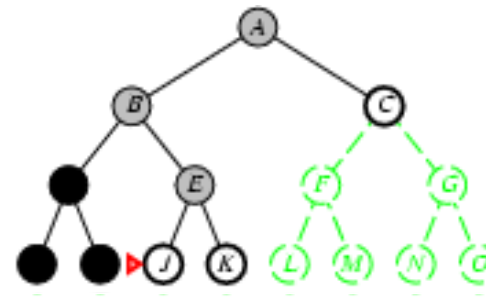
- Complete? Yes (if b is finite)
- Time? $1+b+b^2+b^3+\dots+b^d + b(b^d-1) = O(b^{d+1})$
- Space? $O(b^{d+1})$ (keeps every node in memory)
- Optimal? Yes (if cost = 1 per step)



23

Depth-first search

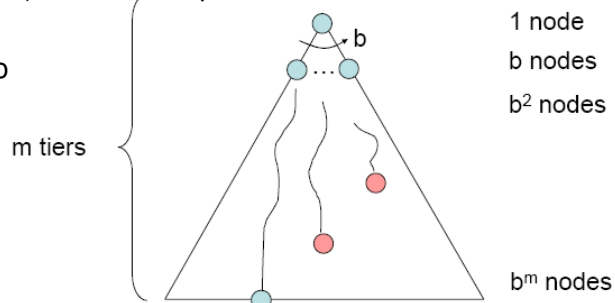
- Expand deepest unexpanded node



24

Depth-first search (con't)

- **Complete?** No: fails in infinite-depth spaces, spaces with loops
 - Modify to avoid repeated states along path → complete in finite spaces
- **Time?** $O(b^m)$: terrible if m is much larger than d
 - but if solutions are dense, may be much faster than breadth-first
- **Space?** $O(bm)$, i.e., linear space!
-
- **Optimal?** No
-



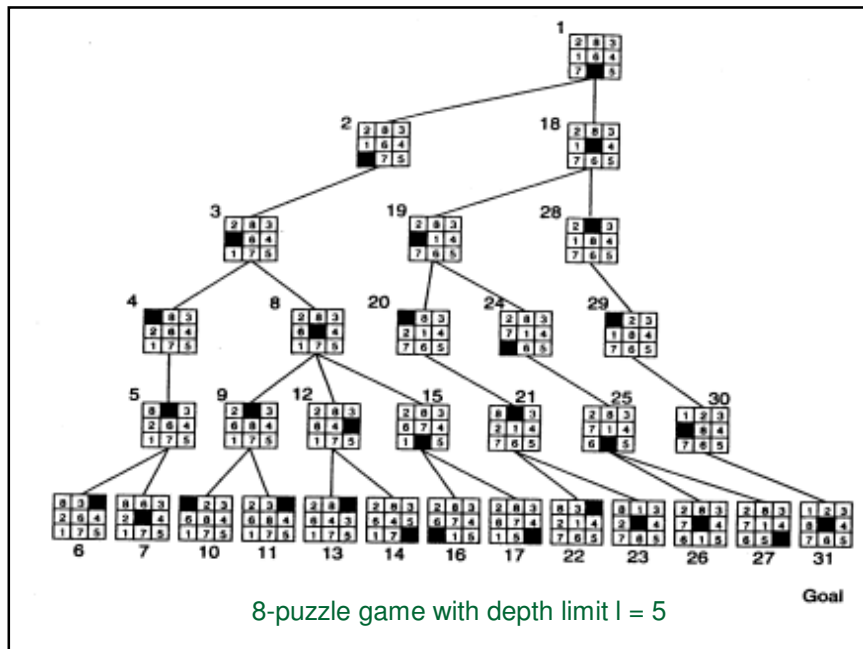
Depth-limited search

- Depth-first search can get stuck on infinite path when a different choice would lead to a solution
 - ⇒ Depth-limited search = depth-first search with depth limit l , i.e., nodes at depth l have no successors

```

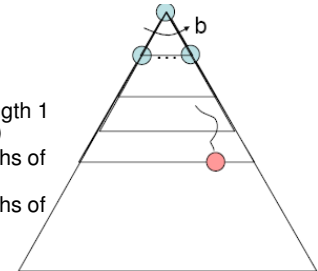
function DEPTH-LIMITED-SEARCH( problem, limit ) returns soln/fail/cutoff
    RECURSIVE-DLS(MAKE-NODE(INITIAL-STATE[problem]), problem, limit)

function RECURSIVE-DLS(node, problem, limit) returns soln/fail/cutoff
    cutoff-occurred? ← false
    if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)
    else if DEPTH[node] = limit then return cutoff
    else for each successor in EXPAND(node, problem) do
        result ← RECURSIVE-DLS(successor, problem, limit)
        if result = cutoff then cutoff-occurred? ← true
        else if result ≠ failure then return result
    if cutoff-occurred? then return cutoff else return failure
    
```



Iterative deepening search

- Problem with depth-limited search: if the shallowest goal is beyond the depth limit, no solution is found.
- ⇒ Iterative deepening search:
 1. Do a DFS which only searches for paths of length 1 or less. (DFS gives up on any path of length 2)
 2. If "1" failed, do a DFS which only searches paths of length 2 or less.
 3. If "2" failed, do a DFS which only searches paths of length 3 or less.
 4.and so on.



function ITERATIVE-DEEPENING-SEARCH(*problem*) **returns** a solution, or failure

inputs: *problem*, a problem

for *depth* $\leftarrow 0$ to ∞ **do**

result $\leftarrow$ DEPTH-LIMITED-SEARCH(*problem*, *depth*)

if *result* $\neq$ cutoff **then return** *result*

Iterative deepening search (con't)

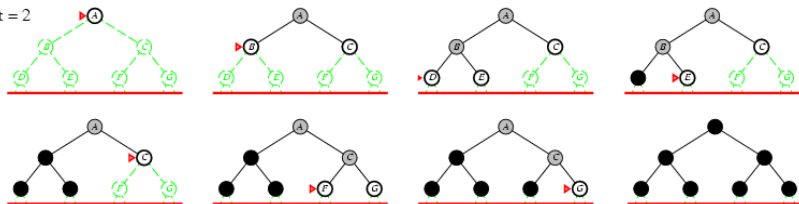
Limit = 0



Limit = 1



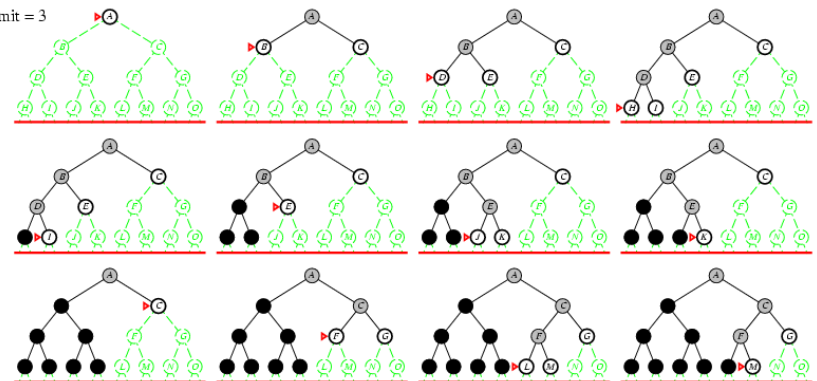
Limit = 2



29

Iterative deepening search (con't)

Limit = 3



30

Iterative deepening search (con't)

- Number of nodes generated in a depth-limited search to depth d with branching factor b :

$$N_{DLS} = b^0 + b^1 + b^2 + \dots + b^{d-2} + b^{d-1} + b^d$$

- Number of nodes generated in an iterative deepening search to depth d with branching factor b :

$$N_{IDS} = (d+1)b^0 + d b^1 + (d-1)b^2 + \dots + 3b^{d-2} + 2b^{d-1} + 1b^d$$

- For $b = 10, d = 5$,
 - $N_{DLS} = 1 + 10 + 100 + 1,000 + 10,000 + 100,000 = 111,111$
 - $N_{IDS} = 6 + 50 + 400 + 3,000 + 20,000 + 100,000 = 123,456$
- Overhead = $(123,456 - 111,111)/111,111 = 11\%$

31

Properties of iterative deepening search

- Complete? Yes
- Time? $(d+1)b^0 + d b^1 + (d-1)b^2 + \dots + b^d = O(b^d)$
- Space? $O(bd)$
- Optimal? Yes, if step cost = 1

32

Summary of algorithms

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete?	Yes	Yes	No	No	Yes
Time	$O(b^{d+1})$	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^l)$	$O(b^d)$
Space	$O(b^{d+1})$	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(bl)$	$O(bd)$
Optimal?	Yes	Yes	No	No	Yes